

Graph Neural Networks for Protein Complex Prediction

A complete guide: from PDB files to complex discovery

Topics covered: GNN fundamentals · Protein graph construction · Protein-protein interaction · Protein complex prediction · PPI-graph-aware embeddings · CORUM-supervised training

1. Graph Neural Networks — Overview

Graph Neural Networks (GNNs) extend deep learning to graph-structured data. Each node aggregates feature information from its neighbours through repeated message-passing rounds, producing rich embeddings that encode both local chemistry and global topology. For proteins this is ideal: atoms or residues become nodes, covalent bonds or spatial contacts become edges.

1.1 Prediction task levels

Task level	Description	Protein example
Node-level	Property per node	Residue type / interface label
Edge-level	Property per node pair	Contact prediction, co-complex
Graph-level	Property for whole graph	Stability, function, complex class

1.2 Core architectures

Architecture	Key idea	Best for
GCN	Spectral convolution	Node / graph classification
GraphSAGE	Sampling-based aggregation	Large-scale graphs
GAT	Attention-weighted neighbours	Heterogeneous graphs
GIN	Maximally expressive WL test	Graph-level tasks
GVP-GNN	Vector + scalar features	3D protein geometry
MPNN	Message passing framework	Molecular property prediction
EGNN	E(n)-equivariant	Rotation-invariant PPI

1.3 Minimal GCN example (PyTorch Geometric)

```
import torch, torch.nn.functional as F
from torch_geometric.nn import GCNConv

class GCN(torch.nn.Module):
    def __init__(self, in_channels, hidden, out_channels):
        super().__init__()
        self.conv1 = GCNConv(in_channels, hidden)
        self.conv2 = GCNConv(hidden, out_channels)

    def forward(self, x, edge_index):
        x = F.relu(self.conv1(x, edge_index))
        x = F.dropout(x, training=self.training)
        return F.log_softmax(self.conv2(x, edge_index), dim=1)
```

1.4 Common pitfalls

- Over-smoothing — too many layers make node embeddings indistinguishable; keep 2–3 layers.
- Data leakage — always split by protein family (sequence identity < 30%), not randomly.
- Imbalanced labels — use `pos_weight` in `BCELoss`; interface residues are only ~5–10% of all residues.
- Scalability — use mini-batching with `DataLoader` for large graphs.

2. Protein Graph Construction from PDB Files

A protein structure from a PDB file can be represented at two granularities:

Graph type	Nodes	Edges	Best for
Residue graph	Amino acids	C α –C α distance < 8 Å	Function, stability, PPI
Atom graph	Heavy atoms	Covalent / vdW bonds	Binding site, drug design

2.1 Building a residue graph

```
import torch, numpy as np
from torch_geometric.data import Data

def build_residue_graph(pdb_coords, residue_features, threshold=8.0):
    # pdb_coords: (N, 3) Ca coordinates
    diff = pdb_coords[:, None] - pdb_coords[None, :]  # (N, N, 3)
    dist = np.linalg.norm(diff, axis=-1)             # (N, N)

    src, dst = np.where((dist < threshold) & (dist > 0))
    edge_index = torch.tensor([src, dst], dtype=torch.long)
    edge_attr = torch.tensor(dist[src, dst], dtype=torch.float).unsqueeze(1)
    x = torch.tensor(residue_features, dtype=torch.float)
    return Data(x=x, edge_index=edge_index, edge_attr=edge_attr)
```

2.2 Node feature table

Feature	Residue graph	Atom graph
Identity	One-hot AA (20-dim)	Atom type (C/N/O/S)
Evolutionary	PSSM, conservation	—
Structural	SASA, secondary struct	Hybridization, aromaticity
Chemical	Hydrophobicity, charge	Formal charge, H-count
Geometric	Backbone dihedrals ϕ, ψ	Bond angles
Deep (recommended)	ESM2 embeddings (1280-dim)	—

2.3 ESM2 embeddings as node features

```
import esm, torch

model, alphabet = esm.pretrained.esm2_t33_650M_UR50D()
batch_converter = alphabet.get_batch_converter()

data = [("protein_A", "MKTLLTLVVVTIVCLDLGY...")]
_, _, tokens = batch_converter(data)

with torch.no_grad():
    results = model(tokens, repr_layers=[33])
```

```
# Shape: (L, 1280) – one 1280-dim vector per residue
esm_embeddings = results["representations"][33][0, 1:-1]
```

3. Protein–Protein Interaction (PPI) Prediction

PPI prediction has two sub-tasks: (a) binary interaction — do two proteins physically bind? and (b) interface prediction — which residues form the contact surface? GNNs are ideal because each protein is a graph, and the interaction lives at the boundary between two graphs.

3.1 Building a combined PPI graph for a pair

```
def build_ppi_graph(coords_A, feats_A, coords_B, feats_B,
                    intra_threshold=8.0, inter_threshold=6.0):
    def intra_edges(coords, threshold):
        diff = coords[:, None] - coords[None, :]
        dist = np.linalg.norm(diff, axis=-1)
        src, dst = np.where((dist < threshold) & (dist > 0))
        return src, dst, dist[src, dst]

    sA, dA, wA = intra_edges(coords_A, intra_threshold)
    sB, dB, wB = intra_edges(coords_B, intra_threshold)

    # Inter-protein interface edges
    diff_AB = coords_A[:, None] - coords_B[None, :]
    dist_AB = np.linalg.norm(diff_AB, axis=-1)
    sI, dI = np.where(dist_AB < inter_threshold)
    wI = dist_AB[sI, dI]
    nA = len(coords_A)

    all_src = np.concatenate([sA, sB + nA, sI, dI + nA])
    all_dst = np.concatenate([dA, dB + nA, dI + nA, sI])
    all_w = np.concatenate([wA, wB, wI, wI])

    edge_index = torch.tensor([all_src, all_dst], dtype=torch.long)
    edge_attr = torch.tensor(all_w, dtype=torch.float).unsqueeze(1)
    x = torch.tensor(np.vstack([feats_A, feats_B]), dtype=torch.float)
    protein_id = torch.cat([torch.zeros(nA), torch.ones(len(coords_B))]).long()
    return Data(x=x, edge_index=edge_index, edge_attr=edge_attr, protein_id=protein_id)
```

3.2 Twin-encoder + cross-attention model

```
from torch_geometric.nn import GATConv, global_mean_pool

class ProteinEncoder(torch.nn.Module):
    def __init__(self, in_dim, hidden=128, heads=4, layers=3):
        super().__init__()
        self.convs = torch.nn.ModuleList([
            GATConv(in_dim if i==0 else hidden,
                    hidden, heads=heads, concat=False, edge_dim=1)
            for i in range(layers)
        ])
        self.norms = torch.nn.ModuleList([torch.nn.LayerNorm(hidden) for _ in range(layers)])

    def forward(self, x, edge_index, edge_attr):
        for conv, norm in zip(self.convs, self.norms):
            x = norm(F.relu(conv(x, edge_index, edge_attr)))
        return x # (N, hidden)
```

```

class CrossAttentionFusion(torch.nn.Module):
    def __init__(self, dim, heads=4):
        super().__init__()
        self.attn = torch.nn.MultiheadAttention(dim, heads, batch_first=True)
        self.norm = torch.nn.LayerNorm(dim)

    def forward(self, h_A, h_B):
        out_A, _ = self.attn(h_A, h_B, h_B)
        out_B, _ = self.attn(h_B, h_A, h_A)
        return self.norm(h_A + out_A), self.norm(h_B + out_B)

class PPIModel(torch.nn.Module):
    def __init__(self, in_dim=1300, hidden=128):
        super().__init__()
        self.encoder = ProteinEncoder(in_dim, hidden)
        self.cross_attn = CrossAttentionFusion(hidden)
        self.interaction_head = torch.nn.Sequential(
            torch.nn.Linear(hidden*2, 64), torch.nn.ReLU(), torch.nn.Linear(64, 1))
        self.interface_head = torch.nn.Linear(hidden, 1)

    def forward(self, x, edge_index, edge_attr, protein_id, batch):
        h = self.encoder(x, edge_index, edge_attr)
        h_A = h[protein_id==0].unsqueeze(0)
        h_B = h[protein_id==1].unsqueeze(0)
        h_A, h_B = self.cross_attn(h_A, h_B)
        h_A, h_B = h_A.squeeze(0), h_B.squeeze(0)
        inter_logit = self.interaction_head(
            torch.cat([h_A.mean(0), h_B.mean(0)], -1))
        h_full = torch.zeros_like(h)
        h_full[protein_id==0] = h_A
        h_full[protein_id==1] = h_B
        return inter_logit, self.interface_head(h_full)

```

3.3 PPI training loop

```

model = PPIModel(in_dim=1300, hidden=128)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=1e-5)

def train_step(batch):
    model.train(); optimizer.zero_grad()
    inter_logit, iface_logits = model(
        batch.x, batch.edge_index, batch.edge_attr,
        batch.protein_id, batch.batch)
    loss_inter = F.binary_cross_entropy_with_logits(
        inter_logit.squeeze(), batch.y_interaction.float())
    loss_iface = F.binary_cross_entropy_with_logits(
        iface_logits.squeeze(), batch.y_interface.float(),
        pos_weight=torch.tensor([10.0]))
    loss = loss_inter + 0.5 * loss_iface
    loss.backward(); optimizer.step()
    return loss.item()

```

3.4 Recommended models and datasets

Model	What it does	Notes
PIPR	Sequence + GNN binary PPI	Chen et al. 2019
GNN-PPI	Graph-level PPI classification	Lv et al. 2021
EGNN-PPI	E(n)-equivariant	Satorras et al.
AlphaFold-Multimer	Structure-based PPI	DeepMind 2022

Dataset	Contents	Use for
STRING	12B+ PPI interactions	Binary prediction
PDBbind	Binding affinities + structure	Affinity regression
DIPS	Dimer structures from PDB	Interface prediction
Negatome	Verified non-interacting pairs	Negative samples
BioGRID	Curated experimental PPI	Training / evaluation

4. Protein Complex Prediction — Multi-chain GNN

Complex prediction extends PPI: you assemble 3 or more chains, predict stoichiometry, and map the full interface network. Two GNNs operate in sequence — one per-chain encoder and one cross-chain heterogeneous GNN that propagates information across all chains jointly.

4.1 Multi-chain heterogeneous graph

```
from torch_geometric.data import HeteroData

def build_complex_graph(chains, intra_cutoff=8.0, inter_cutoff=6.0):
    # chains: {'A': {'coords': (NA,3), 'features': (NA,F)}, 'B': ...}
    data = HeteroData()
    chain_ids = list(chains.keys())

    for cid in chain_ids:
        data[cid].x = torch.tensor(chains[cid]['features'], dtype=torch.float)
        data[cid].coords = torch.tensor(chains[cid]['coords'], dtype=torch.float)

    for cid in chain_ids:
        # intra-chain edges
        coords = chains[cid]['coords']
        dist = np.linalg.norm(coords[:,None]-coords[None,:], axis=-1)
        s, d = np.where((dist < intra_cutoff) & (dist > 0))
        data[cid, 'intra', cid].edge_index = torch.tensor([s,d], dtype=torch.long)
        data[cid, 'intra', cid].edge_attr = torch.tensor(dist[s,d]).unsqueeze(1)

    for i, ci in enumerate(chain_ids):
        # inter-chain edges
        for cj in chain_ids[i+1:]:
            dist = np.linalg.norm(
                chains[ci]['coords'][:,None]-chains[cj]['coords'][None,:], axis=-1)
            s, d = np.where(dist < inter_cutoff)
            w = torch.tensor(dist[s,d]).unsqueeze(1)
            eid = torch.tensor([s,d], dtype=torch.long)
            data[ci, 'inter', cj].edge_index = eid
            data[ci, 'inter', cj].edge_attr = w
            data[cj, 'inter', ci].edge_index = eid.flip(0)
            data[cj, 'inter', ci].edge_attr = w
    return data
```

4.2 Heterogeneous GNN model

```
from torch_geometric.nn import HeteroConv, GATConv

class ComplexGNN(torch.nn.Module):
    def __init__(self, chain_ids, in_dim=29, hidden=128, heads=4, layers=3):
        super().__init__()
        self.chain_ids = chain_ids
        self.input_proj = torch.nn.ModuleDict(
            {cid: torch.nn.Linear(in_dim, hidden) for cid in chain_ids})
        self.hetero_convs = torch.nn.ModuleList()
        for _ in range(layers):
            conv_dict = {}
            for cid in chain_ids:
                conv_dict[cid, 'intra', cid] = GATConv(
```

```

        hidden, hidden, heads=heads, concat=False,
        edge_dim=1, add_self_loops=False)
    for i,ci in enumerate(chain_ids):
        for cj in chain_ids[i+1:]:
            for src,dst in [(ci,cj),(cj,ci)]:
                conv_dict[src,'inter',dst] = GATConv(
                    (hidden,hidden), hidden, heads=heads,
                    concat=False, edge_dim=1, add_self_loops=False)
            self.hetero_convs.append(HeteroConv(conv_dict, aggr='sum'))
    self.stoich_head = torch.nn.Linear(hidden, len(chain_ids))
    self.complex_head = torch.nn.Linear(hidden, hidden)

def forward(self, x_dict, edge_index_dict, edge_attr_dict):
    h = {cid: F.relu(self.input_proj[cid](x_dict[cid]))
         for cid in self.chain_ids}
    for conv in self.hetero_convs:
        h_new = conv(h, edge_index_dict, edge_attr_dict)
        h = {cid: F.relu(h_new.get(cid, h[cid])) + h[cid]
             for cid in self.chain_ids}
    complex_emb = torch.stack(
        [h[cid].mean(0) for cid in self.chain_ids]).mean(0)
    return h, self.complex_head(complex_emb), self.stoich_head(complex_emb)

```

5. Your Complete Pipeline: PDB Graphs + PPI Network + CORUM

You have: (1) PyG graph objects for each protein derived from PDB files, (2) a PPI graph connecting those proteins, and (3) known complexes from CORUM. The following pipeline encodes each protein's structure with a GNN, enriches those embeddings with PPI context via a second GNN, then trains complex prediction supervised by CORUM labels.

5.1 Step 1 — Protein structure GNN encoder

```
class ProteinStructureGNN(torch.nn.Module):
    def __init__(self, in_dim, hidden=128, out_dim=64, heads=4):
        super().__init__()
        self.conv1 = GATConv(in_dim, hidden, heads=heads, concat=False, edge_dim=1)
        self.conv2 = GATConv(hidden, hidden, heads=heads, concat=False, edge_dim=1)
        self.conv3 = GATConv(hidden, out_dim, heads=heads, concat=False, edge_dim=1)
        self.norm1 = torch.nn.LayerNorm(hidden)
        self.norm2 = torch.nn.LayerNorm(hidden)

    def forward(self, x, edge_index, edge_attr, batch):
        x = self.norm1(F.relu(self.conv1(x, edge_index, edge_attr)))
        x = self.norm2(F.relu(self.conv2(x, edge_index, edge_attr)))
        x = self.conv3(x, edge_index, edge_attr)
        return global_mean_pool(x, batch) # (num_proteins, out_dim)

# Encode all proteins and store embeddings
structure_gnn = ProteinStructureGNN(in_dim=YOUR_FEAT_DIM)
structure_gnn.eval()
protein_embeddings = {}
loader = DataLoader([g for _, g in protein_graphs], batch_size=32)
ids = [pid for pid, _ in protein_graphs]
with torch.no_grad():
    all_embeddings = []
    for batch in loader:
        emb = structure_gnn(batch.x, batch.edge_index, batch.edge_attr, batch.batch)
        all_embeddings.append(emb)
all_embeddings = torch.cat(all_embeddings, dim=0)
for pid, emb in zip(ids, all_embeddings):
    protein_embeddings[pid] = emb
```

5.2 Step 2 — Build PPI network as a PyG graph

```
def build_ppi_network_graph(ppi_edges, protein_embeddings):
    proteins = sorted(protein_embeddings.keys())
    idx_map = {pid: i for i, pid in enumerate(proteins)}
    x = torch.stack([protein_embeddings[p] for p in proteins]) # (N, D)
    src = [idx_map[a] for a, b in ppi_edges if a in idx_map and b in idx_map]
    dst = [idx_map[b] for a, b in ppi_edges if a in idx_map and b in idx_map]
    edge_index = torch.tensor([src+dst, dst+src], dtype=torch.long) # bidirectional
    return Data(x=x, edge_index=edge_index), proteins, idx_map
```

5.3 Step 3 — Load CORUM labels

```
import pandas as pd

def load_corum_labels(corum_file, idx_map):
    df = pd.read_csv(corum_file, sep='\t')
    complex_list = []
    node_complex = {i: [] for i in range(len(idx_map))}
    for cid, row in enumerate(df.itertuples()):
        members_raw = row.subunits_Gene_name.split(';')
        members_idx = [idx_map[p] for p in members_raw if p in idx_map]
        if len(members_idx) >= 2:
            complex_list.append(set(members_idx))
            for m in members_idx:
                node_complex[m].append(cid)
    return complex_list, node_complex

def edge_cocomplex_labels(edge_index, complex_list):
    labels = []
    for s, d in edge_index.T.tolist():
        shared = any(s in cx and d in cx for cx in complex_list)
        labels.append(1 if shared else 0)
    return torch.tensor(labels, dtype=torch.float)
```

5.4 Step 4 — PPI network GNN (context enrichment)

```
from torch_geometric.nn import SAGEConv

class PPINetworkGNN(torch.nn.Module):
    def __init__(self, in_dim=64, hidden=128, out_dim=64, layers=3):
        super().__init__()
        self.convs = torch.nn.ModuleList()
        self.norms = torch.nn.ModuleList()
        for i in range(layers):
            d_in = in_dim if i == 0 else hidden
            d_out = out_dim if i == layers-1 else hidden
            self.convs.append(SAGEConv(d_in, d_out))
            self.norms.append(torch.nn.LayerNorm(d_out))

    def forward(self, x, edge_index):
        for conv, norm in zip(self.convs, self.norms):
            x = norm(F.relu(conv(x, edge_index)))
        return x # (N_proteins, out_dim)
```

5.5 Step 5 — Edge-level co-complex prediction head

```
class ComplexEdgePredictor(torch.nn.Module):
    def __init__(self, dim=64):
        super().__init__()
        self.mlp = torch.nn.Sequential(
            torch.nn.Linear(dim*2, 128), torch.nn.ReLU(),
            torch.nn.Linear(128, 64), torch.nn.ReLU(),
            torch.nn.Linear(64, 1))

    def forward(self, h, edge_index):
```

```

src, dst = edge_index
pairs = torch.cat([h[src], h[dst]], dim=-1) # (E, 2*dim)
return self.mlp(pairs).squeeze(-1)          # (E,) logits

```

5.6 Step 6 — Full training loop

```

structure_gnn = ProteinStructureGNN(in_dim=YOUR_FEAT_DIM, out_dim=64)
ppi_gnn = PPINetworkGNN(in_dim=64, hidden=128, out_dim=64)
edge_predictor = ComplexEdgePredictor(dim=64)
optimizer = torch.optim.Adam(
    list(structure_gnn.parameters()) +
    list(ppi_gnn.parameters()) +
    list(edge_predictor.parameters()), lr=1e-3)

ppi_data, proteins, idx_map = build_ppi_network_graph(ppi_edges, protein_embeddings)
complex_list, _ = load_corum_labels('corum.tsv', idx_map)
edge_labels = edge_cocomplex_labels(ppi_data.edge_index, complex_list)
pos_weight = torch.tensor([(edge_labels==0).sum() / (edge_labels==1).sum()])

for epoch in range(200):
    structure_gnn.train(); ppi_gnn.train(); edge_predictor.train()
    optimizer.zero_grad()

    all_embeds = []
    for batch in protein_loader:
        emb = structure_gnn(batch.x, batch.edge_index, batch.edge_attr, batch.batch)
        all_embeds.append(emb)
    x_struct = torch.cat(all_embeds) # (N, 64)
    h = ppi_gnn(x_struct, ppi_data.edge_index) # (N, 64)
    logits = edge_predictor(h, ppi_data.edge_index)

    loss = F.binary_cross_entropy_with_logits(
        logits, edge_labels, pos_weight=pos_weight)
    loss.backward(); optimizer.step()
    if epoch % 20 == 0:
        print(f"Epoch {epoch:3d} | Loss {loss.item():.4f}")

```

6. Inference — Discovering Complexes from Embeddings

6.1 Louvain community detection on scored PPI graph

```
import networkx as nx
from community import best_partition # pip install python-louvain

def predict_complexes_louvain(ppi_edges, edge_scores, threshold=0.5):
    G = nx.Graph()
    scores = torch.sigmoid(edge_scores).detach().numpy()
    for (s, d), sc in zip(zip(*ppi_edges), scores):
        if sc > threshold:
            G.add_edge(int(s), int(d), weight=float(sc))
    partition = best_partition(G, weight='weight')
    complexes = {}
    for node, comm in partition.items():
        complexes.setdefault(comm, []).append(node)
    return list(complexes.values())
```

6.2 Spectral clustering on protein embeddings

```
from sklearn.cluster import SpectralClustering

def predict_complexes_embedding(h, n_clusters=50):
    h_np = h.detach().numpy()
    clust = SpectralClustering(n_clusters=n_clusters,
                              affinity='nearest_neighbors', random_state=42)
    labels = clust.fit_predict(h_np)
    complexes = {}
    for i, l in enumerate(labels):
        complexes.setdefault(int(l), []).append(i)
    return list(complexes.values())
```

6.3 Evaluation against CORUM ground truth

```
def jaccard(set_a, set_b):
    a, b = set(set_a), set(set_b)
    return len(a & b) / len(a | b) if a | b else 0.0

def evaluate_complexes(predicted, ground_truth, threshold=0.25):
    matched = sum(
        1 for gt in ground_truth
        if max((jaccard(gt, pred) for pred in predicted), default=0) >= threshold)
    recall = matched / len(ground_truth)
    precision = sum(
        1 for pred in predicted
        if max((jaccard(pred, gt) for gt in ground_truth), default=0) >= threshold
    ) / len(predicted)
    f1 = 2 * precision * recall / (precision + recall + 1e-8)
    print(f"Precision: {precision:.3f} Recall: {recall:.3f} F1: {f1:.3f}")
    return precision, recall, f1
```

6.4 Evaluation metrics reference

Metric	Formula	What it measures
Jaccard index	$ A \cap B / A \cup B $	Overlap between two complexes
MMR (precision)	% predicted complexes matched	Predicted quality
Recall	% true complexes matched	Coverage of known complexes
F1	$2PR/(P+R)$	Harmonic mean
AUC-ROC	Area under ROC curve	Edge co-complex ranking

7. Practical Recommendations & State-of-the-Art

7.1 Key tips for strong performance

- Negative sampling matters — true non-interacting pairs are rare. Use Negatome + randomly shuffled pairs at 1:3 (pos:neg) ratio.
- Data leakage — split by protein family using sequence identity < 30%, not randomly. Random splits inflate accuracy by ~15–20%.
- Geometric features — GVP-GNN style backbone frame orientation vectors consistently beat feature-only models.
- ESM2 node features — esm2_t33_650M embeddings give a strong baseline even with a simple 2-layer GCN on top.
- Train structure GNN and PPI GNN jointly — end-to-end gradients flow from complex loss back into structure encoder.
- Two-stage approach — use AlphaFold-Multimer for structure prediction, then GNN downstream for property prediction when labeled data is scarce.
- CORUM class imbalance — most protein pairs do NOT form a complex. Use pos_weight or focal loss.

7.2 State-of-the-art methods

Method	Strategy	Strength
AlphaFold-Multimer	Evoformer + structure module	Best accuracy overall
ESMFold + DockQ	Sequence → structure → quality	Fast, no MSA needed
EquiDock	SE(3)-equivariant rigid docking	Rigid-body docking
DiffDock-PP	Diffusion over SE(3)	Generative docking
GNN-Complex	Heterogeneous GNN	Interpretable, modular
HDOCK	Template + graph hybrid	Flexible docking

7.3 Complex prediction datasets

Dataset	Size	Use for
PDB biological assemblies	~170K complexes	Structure supervision
CORUM	~4K complexes	Mammalian complex labels
QSBio	~100K assemblies	Stoichiometry prediction
Interactome3D	Structural PPIs	Interface training
AlphaFold-Multimer preds	Millions	Pseudo-labelled training
CASP-CAPRI	~200 targets	Benchmark evaluation

7.4 Component summary

Component	Input	Output
ProteinStructureGNN	PDB PyG graph per protein	64-dim structure embedding
PPINetworkGNN	PPI graph + structure embeds	64-dim context embedding
ComplexEdgePredictor	Protein pair embeddings	Co-complex probability
Louvain / Spectral	Weighted edge scores	Predicted complex clusters
CORUM evaluator	Predicted vs known complexes	Precision, Recall, F1

Generated from the GNN for Protein Complex Prediction chat session.